

Etapa 3 - Testes e Documentação

Sistema de Locadora de Veículos - API REST

3.1. Integração do Swagger ao Projeto

A documentação e os testes das APIs foram realizados por meio do Swagger (Swashbuckle.AspNetCore), integrado ao projeto ASP.NET Core.

A integração foi feita no arquivo Program.cs com os serviços de geração do OpenAPI e da interface interativa:

```
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

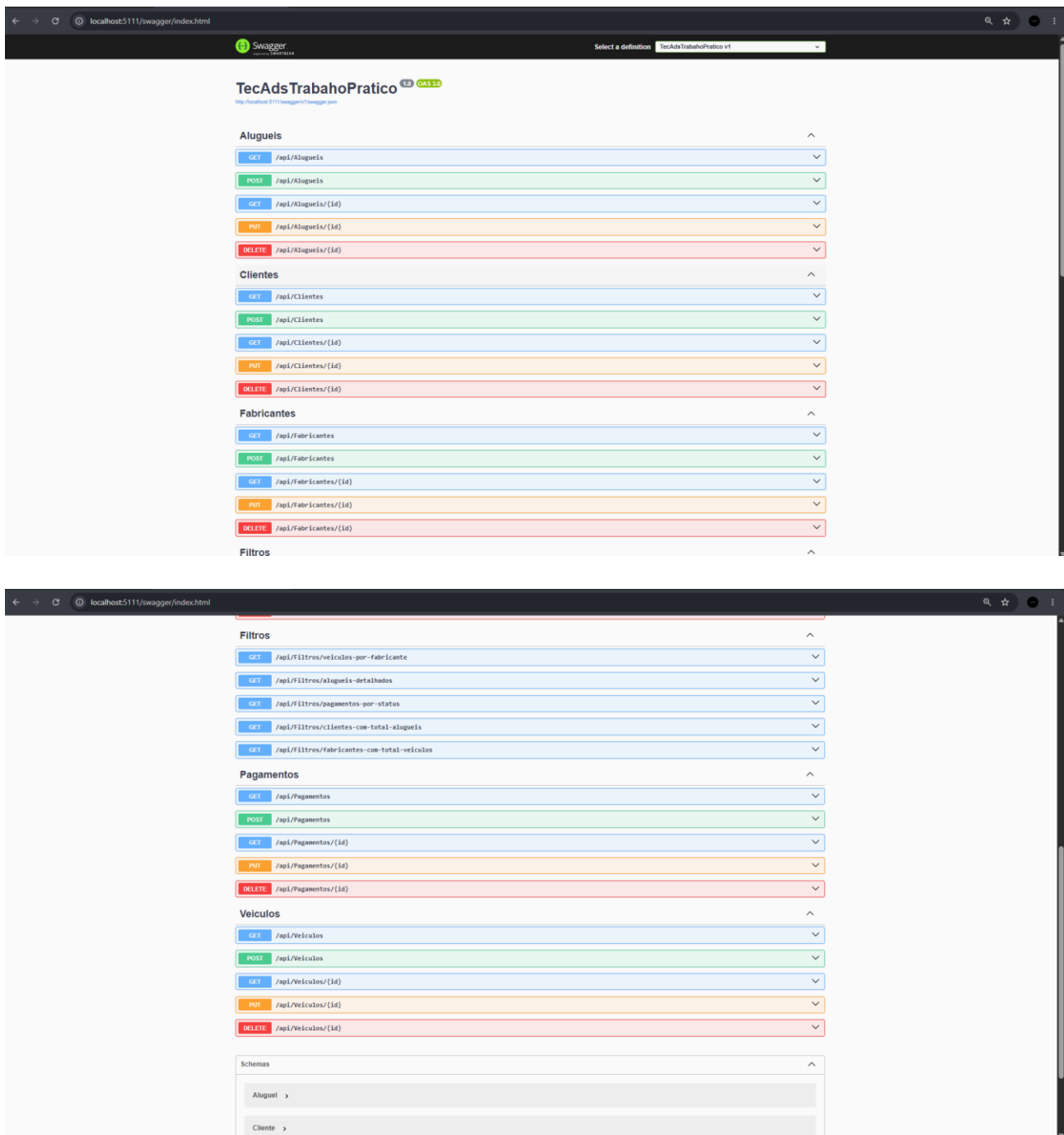
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

O pacote utilizado está declarado no arquivo .csproj:

```
<PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
```

Com a aplicação em execução (dotnet run), a interface do Swagger fica disponível na URL:

<http://localhost:5111/swagger>



3.2. Documentação dos Endpoints

A API é composta por seis grupos de endpoints. Cinco deles implementam operações CRUD completas (GET, GET por id, POST, PUT, DELETE) sobre as entidades do domínio, e um grupo (Filtros) oferece consultas específicas.

Entidades e seus campos

Entidade	Campos
Fabricante	IdFabricante, Nome, Descrição, País
Veículo	IdVeículo, IdFabricante, Modelo, AnoFabricação, Quilometragem, Status
Cliente	IdCliente, Nome, Cpf, Email, Telefone, Endereço

Aluguel	IdAluguel, IdCliente, IdVeiculo, DataInicio, DataFim, DataDevolucao, QuilometragemInicial, QuilometragemFinal, ValorDiaria, ValorTotal
Pagamento	IdPagamento, IdAluguel, Valor, DataPagamento, FormaPagamento, Status

Padrão de endpoints CRUD

Os controllers Fabricantes, Veiculos, Clientes, Alugueis e Pagamentos seguem o mesmo padrão REST. Substituindo {recurso} pelo nome do recurso:

Método HTTP	Rota	Descrição	Parâmetros	Códigos de resposta
GET	/api/{recurso}	Lista todos os registros	Nenhum	200 OK
GET	/api/{recurso}/{id}	Retorna um registro por id	id (path, int)	200 OK, 404 Not Found
POST	/api/{recurso}	Cria um novo registro	Objeto JSON no corpo	201 Created
PUT	/api/{recurso}/{id}	Atualiza um registro existente	id (path, int) + objeto JSON	204 No Content, 400 Bad Request
DELETE	/api/{recurso}/{id}	Remove um registro	id (path, int)	204 No Content, 404 Not Found

Endpoints de Filtros (/api/Filtros)

Método	Rota	Parâmetro	Resposta
GET	/api/Filtros/veiculos-por-fabricante	fabricante (query, opcional)	200 OK
GET	/api/Filtros/alugueis-detalhados	Nenhum	200 OK
GET	/api/Filtros/pagamentos-por-status	status (query, opcional)	200 OK
GET	/api/Filtros/clientes-com-total-alugueis	Nenhum	200 OK
GET	/api/Filtros/fabricantes-com-total-veiculos	Nenhum	200 OK

3.3. Relatório de Testes Manuais via Swagger

Os testes foram executados pela interface do Swagger, respeitando a ordem de dependência entre as entidades (Fabricante -> Cliente -> Veículo -> Aluguel -> Pagamento -> Filtros). Para cada teste são apresentadas a chamada realizada (método, rota e corpo) e o retorno obtido.

Teste 1 - POST Fabricante (criar)

Rota: POST /api/Fabricantes

Corpo da requisição:

```
{
  "nome": "Toyota",
  "descricao": "Montadora japonesa",
  "pais": "Japão"
}
```

Resultado esperado: 201 Created, retornando o fabricante criado com idFabricante preenchido.

The screenshot shows the Swagger UI interface in a web browser at localhost:5111/swagger/index.html. The 'Execute' tab is active, displaying the details of a successful POST request. The request body is a JSON object: {"nome": "Toyota", "descricao": "Montadora japonesa", "pais": "Japão"}. The response is a 201 status code with a response body: {"idFabricante": 3, "nome": "Toyota", "descricao": "Montadora japonesa", "pais": "Japão", "veiculos": []}. The response headers include content-type: application/json; charset=utf-8, date: Sun, 04 Oct 2026 23:12:37 GMT, location: http://localhost:5111/api/Fabricantes/3, server: Kestrel, and transfer-encoding: chunked. A table at the bottom shows the response code 200 and description OK.

Teste 2 - GET Fabricantes (listar todos)

Rota: GET /api/Fabricantes

Resultado esperado: 200 OK com a lista contendo o fabricante criado.

The screenshot shows the Swagger UI interface in a web browser at localhost:5111/swagger/index.html. The 'Execute' tab is active, displaying the details of a successful GET request. The request body is empty. The response is a 200 status code with a response body: [{"idFabricante": 3, "nome": "Toyota", "descricao": "Montadora japonesa", "pais": "Japão", "veiculos": []}]. The response headers include content-type: application/json; charset=utf-8, date: Sun, 04 Oct 2026 23:12:39 GMT, server: Kestrel, and transfer-encoding: chunked. A table at the bottom shows the response code 200 and description OK.

Teste 3 - GET Fabricante por id

Rota: GET /api/Fabricantes/3

Resultado esperado: 200 OK com os dados do fabricante de id 3.

The screenshot shows the Swagger UI interface in a web browser. The URL bar indicates the path is `localhost:5111/swagger/index.html`. The input field for the `id` parameter is set to `3`. The `Execute` button is highlighted. Below the input, the `Responses` section shows a `200` status code with a `OK` description. The `Response body` is displayed as a JSON object:

```
{  "idFabricante": 3,  "nome": "Toyota",  "descricao": "Montadora japonesa",  "pais": "Japão",  "veiculos": []}
```

. The `Response headers` section shows:

```
content-type: application/json; charset=utf-8  date: Sun, 04 Oct 2026 23:13:33 GMT  server: Kestrel  transfer-encoding: chunked
```

. The bottom table shows a `200` status code with a `OK` description and no links.

Teste 4 - GET Fabricante inexistente (validação de 404)

Rota: GET `/api/Fabricantes/999`

Resultado esperado: 404 Not Found.

The screenshot shows the Swagger UI interface. The input field for the `id` parameter is set to `999`. The `Execute` button is highlighted. Below the input, the `Responses` section shows a `404` status code with an `Error: Not Found` description. The `Response body` is displayed as a JSON object:

```
{  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.5",  "title": "Not Found",  "status": 404,  "traceId": "00-170834c5c80d08a1e8b906c3feb9f3-33fc3cb323629a46-00"}
```

. The `Response headers` section shows:

```
content-type: application/problem+json; charset=utf-8  date: Sun, 04 Oct 2026 23:14:03 GMT  server: Kestrel  transfer-encoding: chunked
```

. The bottom table shows a `200` status code with a `OK` description and no links.

Teste 5 - PUT Fabricante (atualizar)

Rota: PUT `/api/Fabricantes/3`

Corpo da requisição:

```
{  "idFabricante": 3,  "nome": "Toyota do Brasil",  "descricao": "Montadora japonesa com fábrica no Brasil",  "pais": "Japão"}
```

```
}
```

Resultado esperado: 204 No Content.

The screenshot shows the Swagger UI interface in a web browser at localhost:5111/swagger/index.html. The 'Execute' button is highlighted in blue. Below it, the 'Responses' section is expanded, showing the details of the 204 response. The 'Curl' section displays the command used to make the request. The 'Request URL' is 'http://localhost:5111/api/Fabricantes/3'. The 'Server response' section shows the status code '204' and the response headers: 'date: Sun, 04 Oct 2026 23:15:57 GMT' and 'server: Kestrel'. The 'Responses' table at the bottom shows a single entry for status code 200 with description 'OK' and no links.

```
curl -X 'POST' \
  'http://localhost:5111/api/Fabricantes/3' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "idFabricante": 3,
    "nome": "Toyota do Brasil",
    "descricao": "Montadora japonesa com fábrica no Brasil",
    "pais": "Japão"
  }'
```

Request URL
http://localhost:5111/api/Fabricantes/3

Server response

Code	Details
204	Response headers date: Sun, 04 Oct 2026 23:15:57 GMT server: Kestrel

Responses

Code	Description	Links
200	OK	No links

Teste 6 - POST Cliente (criar)

Rota: POST /api/Clientes

Corpo da requisição:

```
{
  "nome": "Felippe Salvo",
  "cpf": "12345678900",
  "email": "felippe.salvo@email.com",
  "telefone": "31999990000",
  "endereco": "Rua das Flores, 123 - Belo Horizonte/MG"
}
```

Resultado esperado: 201 Created com idCliente preenchido.

The screenshot shows the Swagger UI interface in a web browser at localhost:5111/swagger/index.html. The 'Execute' button is highlighted in blue. Below it, the 'Responses' section is expanded, showing the details of the 201 response. The 'Curl' section displays the command used to make the request. The 'Request URL' is 'http://localhost:5111/api/Clientes'. The 'Server response' section shows the status code '201' and the response body: '{ "idCliente": 1, "nome": "Felippe Salvo", "cpf": "12345678900", "email": "felippe.salvo@email.com", "telefone": "31999990000", "endereco": "Rua das Flores, 123 - Belo Horizonte/MG", "aluguel": [] }'. The 'Response headers' section shows: 'content-type: application/json; charset=utf-8', 'date: Sun, 04 Oct 2026 23:17:00 GMT', 'location: http://localhost:5111/api/Clientes/1', 'server: Kestrel', and 'transfer-encoding: chunked'. The 'Responses' table at the bottom shows a single entry for status code 200 with description 'OK' and no links.

```
curl -X 'POST' \
  'http://localhost:5111/api/Clientes' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
    "nome": "Felippe Salvo",
    "cpf": "12345678900",
    "email": "felippe.salvo@email.com",
    "telefone": "31999990000",
    "endereco": "Rua das Flores, 123 - Belo Horizonte/MG"
  }'
```

Request URL
http://localhost:5111/api/Clientes

Server response

Code	Details
201	Response body { "idCliente": 1, "nome": "Felippe Salvo", "cpf": "12345678900", "email": "felippe.salvo@email.com", "telefone": "31999990000", "endereco": "Rua das Flores, 123 - Belo Horizonte/MG", "aluguel": [] }

Response headers
content-type: application/json; charset=utf-8
date: Sun, 04 Oct 2026 23:17:00 GMT
location: http://localhost:5111/api/Clientes/1
server: Kestrel
transfer-encoding: chunked

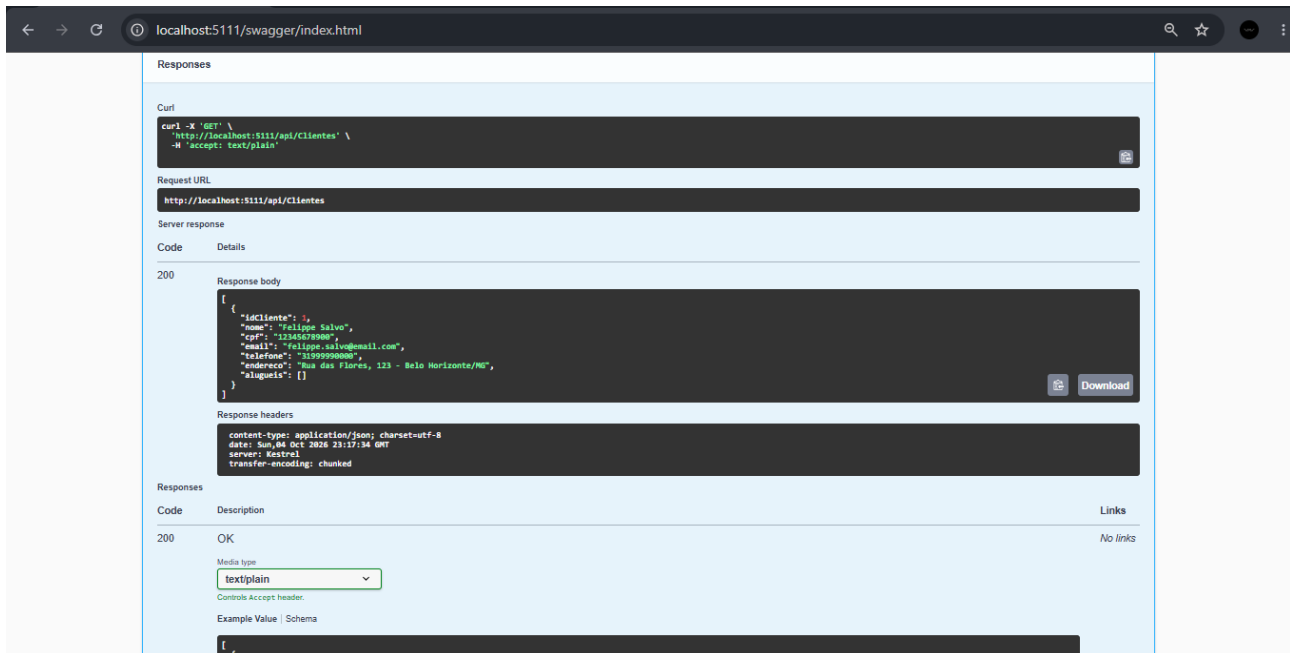
Responses

Code	Description	Links
200	OK	No links

Teste 7 - GET Clientes (listar todos)

Rota: GET /api/Clientes

Resultado esperado: 200 OK com a lista de clientes.



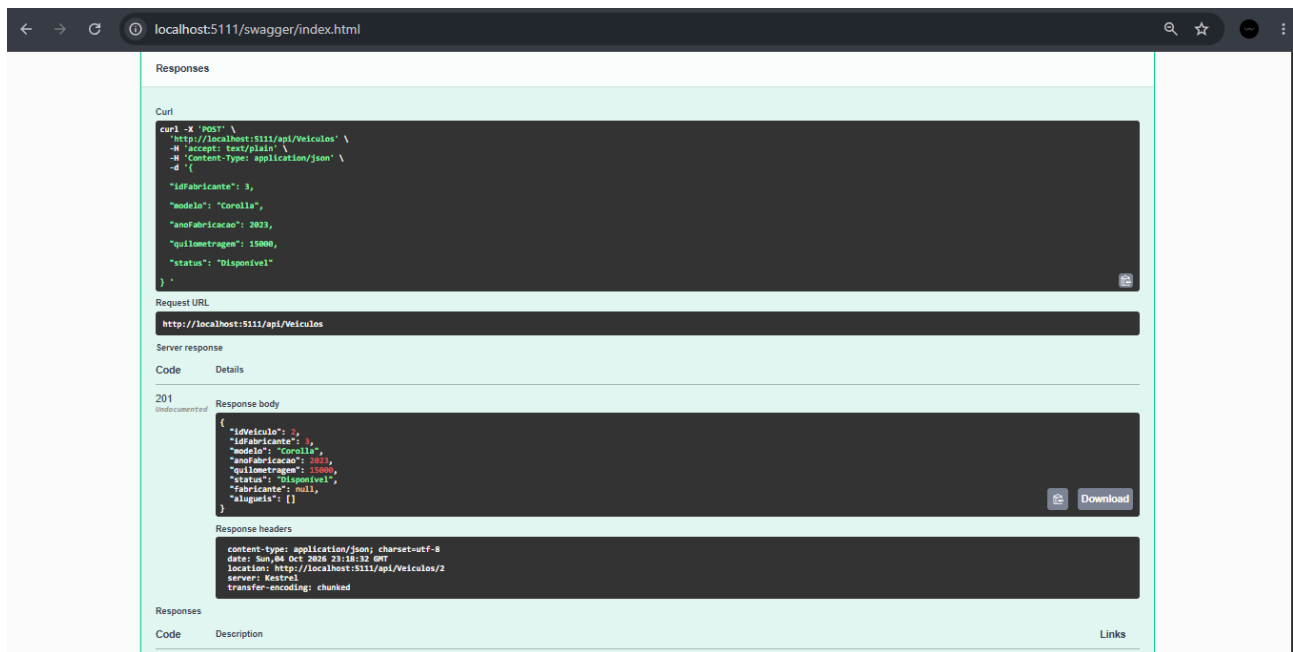
Teste 8 - POST Veículo (criar)

Rota: POST /api/Veiculos

Corpo da requisição:

```
{
  "idFabricante": 3,
  "modelo": "Corolla",
  "anoFabricacao": 2023,
  "quilometragem": 15000,
  "status": "Disponível"
}
```

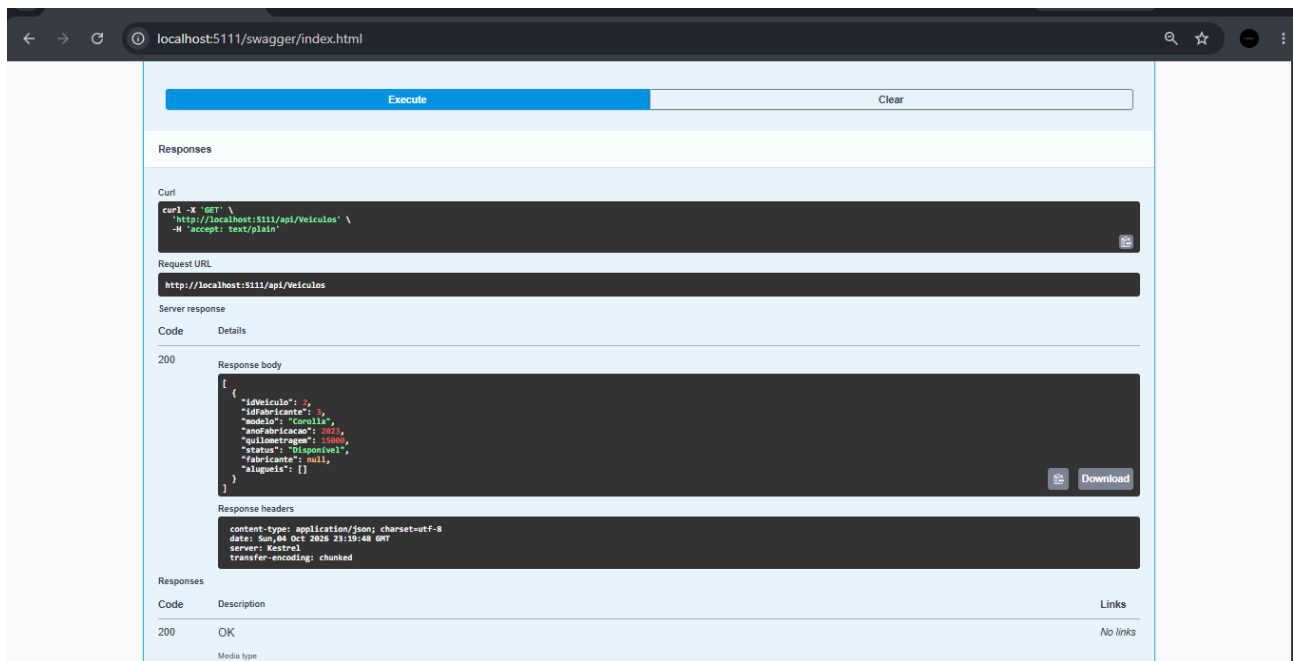
Resultado esperado: 201 Created com idVeiculo preenchido.



Teste 9 - GET Veículos (listar todos)

Rota: GET `/api/Veiculos`

Resultado esperado: 200 OK com a lista de veículos.



Teste 10 - POST Aluguel (criar)

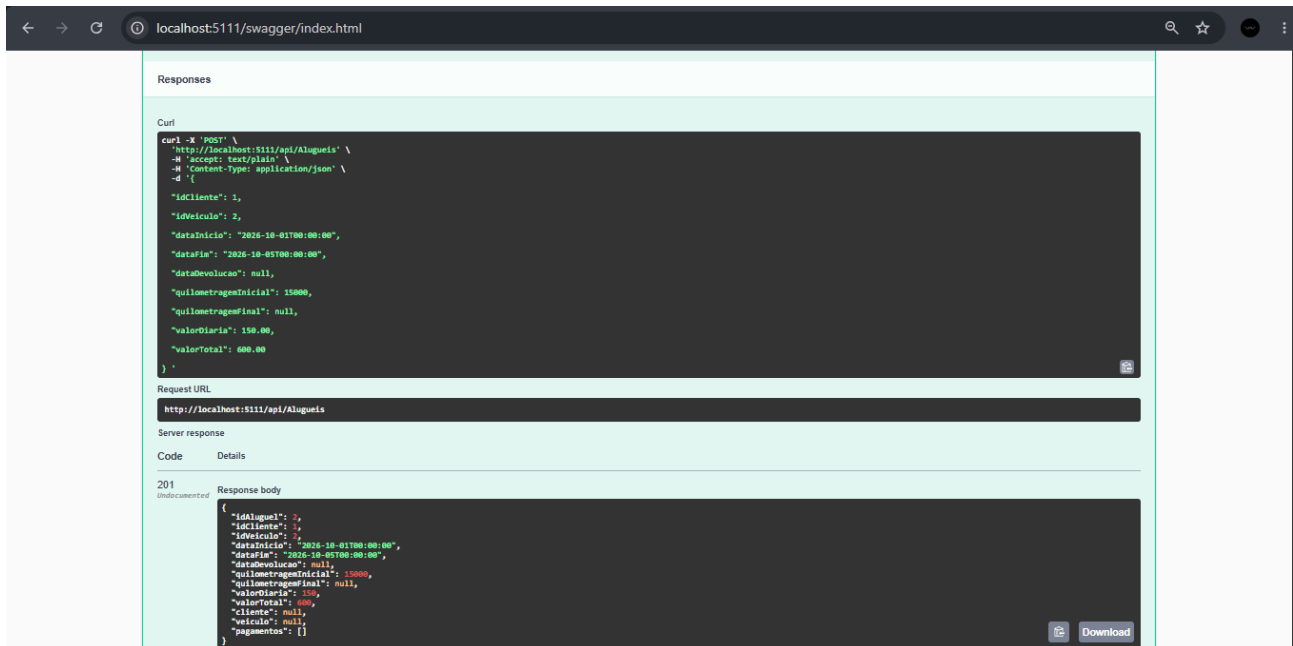
Rota: POST `/api/Alugueis`

Corpo da requisição:

```
{
  "idCliente": 1,
  "idVeiculo": 2,
  "dataInicio": "2026-10-01T00:00:00",
  "dataFim": "2026-10-05T00:00:00",
  "dataDevolucao": null,
}
```

```
"quilometragemInicial": 15000,  
"quilometragemFinal": null,  
"valorDiaria": 150.00,  
"valorTotal": 600.00  
}
```

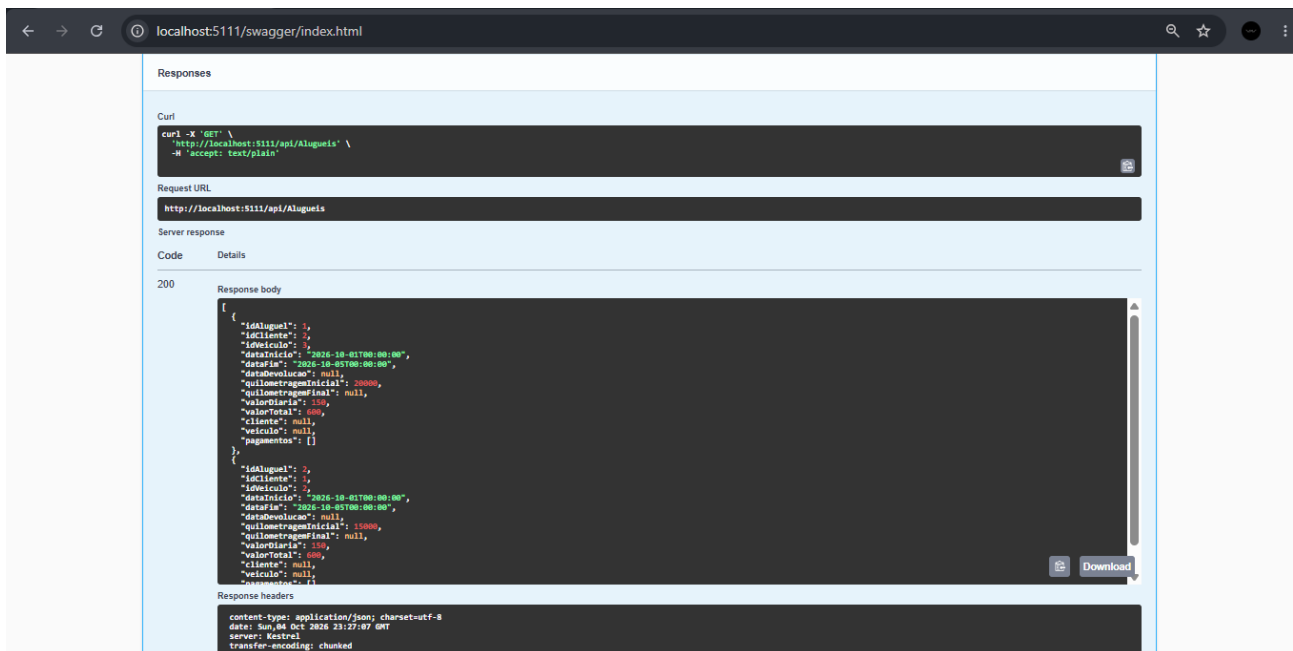
Resultado esperado: 201 Created com idAluguel preenchido.



Teste 11 - GET Aluguéis (listar todos)

Rota: GET /api/Alugueis

Resultado esperado: 200 OK com a lista de alugueis.



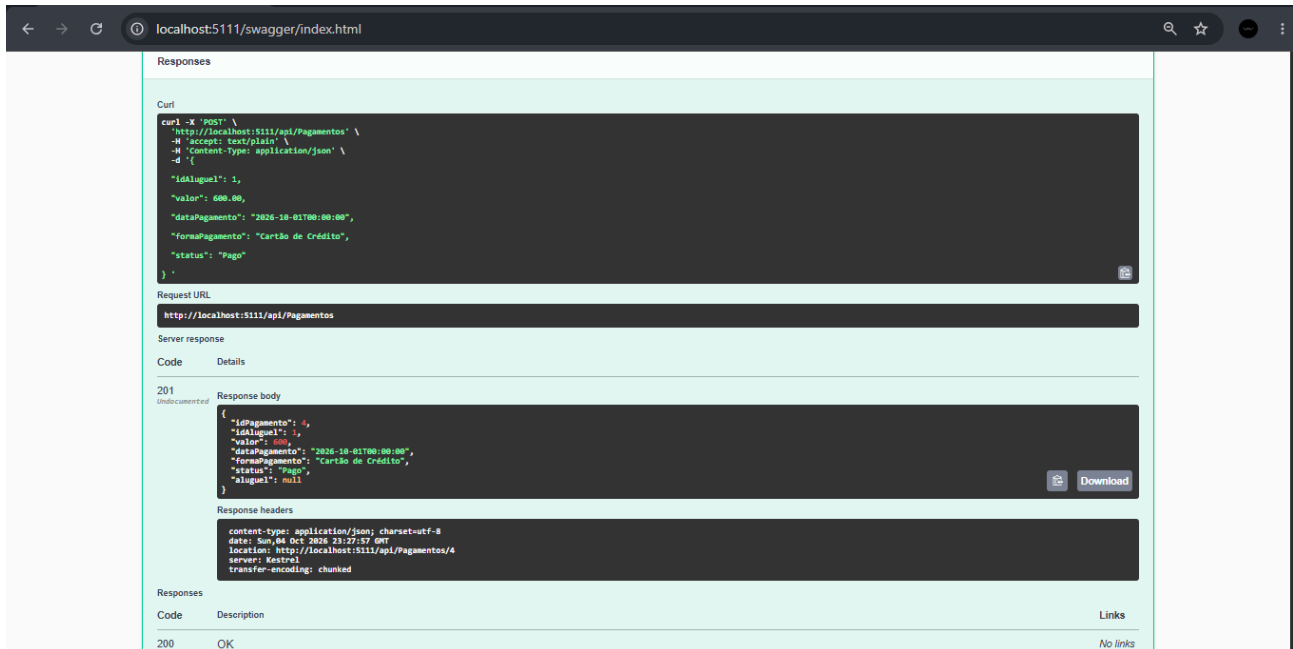
Teste 12 - POST Pagamento (criar)

Rota: POST /api/Pagamentos

Corpo da requisição:

```
{
  "idAluguel": 1,
  "valor": 600.00,
  "dataPagamento": "2026-10-01T00:00:00",
  "formaPagamento": "Cartão de Crédito",
  "status": "Pago"
}
```

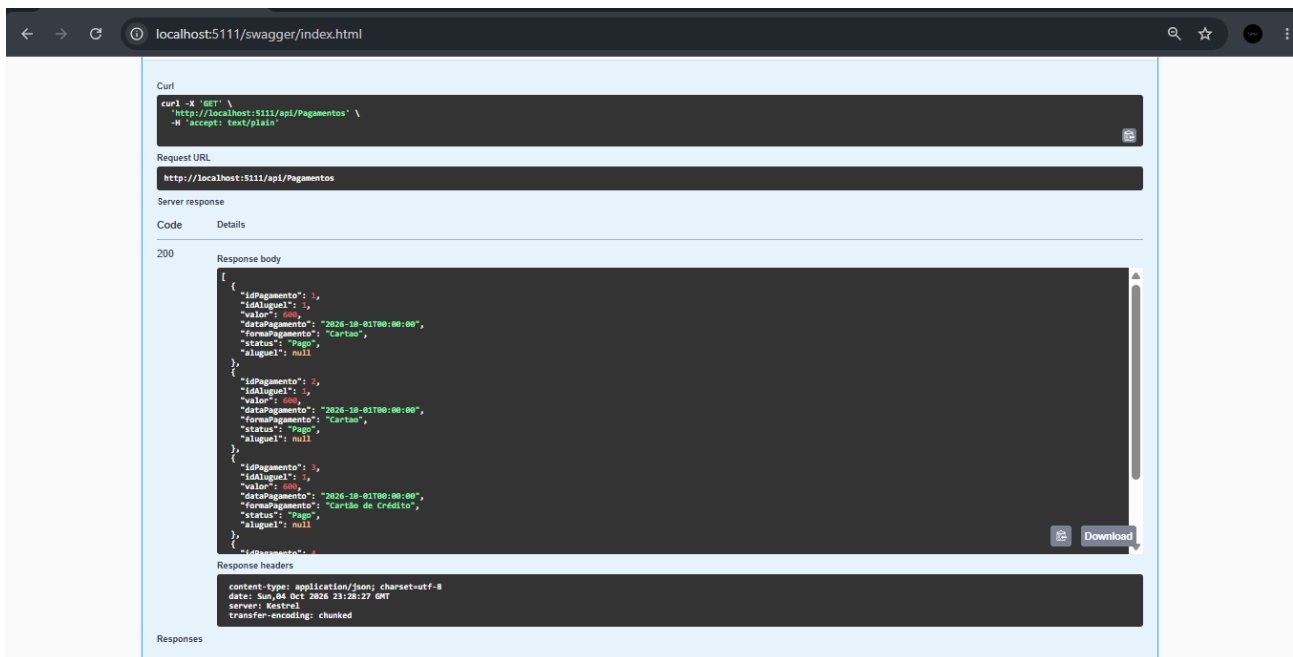
Resultado esperado: 201 Created com idPagamento preenchido.



Teste 13 - GET Pagamentos (listar todos)

Rota: GET /api/Pagamentos

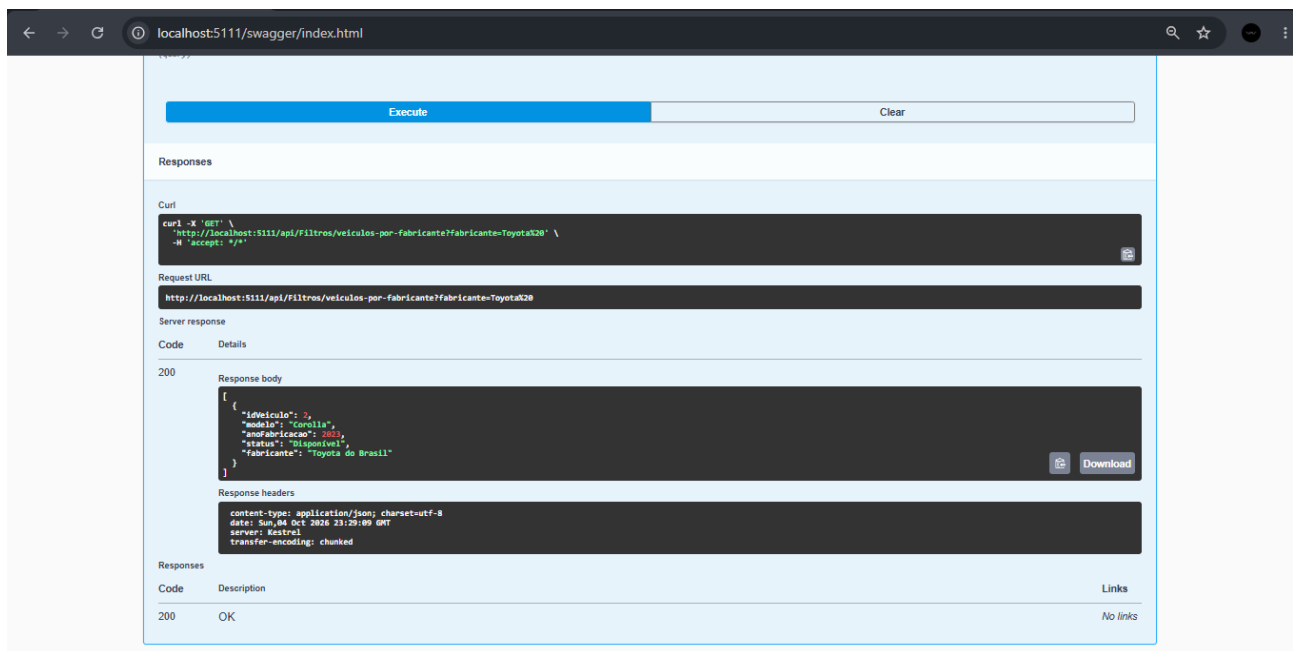
Resultado esperado: 200 OK com a lista de pagamentos.



Teste 14 - Filtro: veículos por fabricante

Rota: GET /api/Filtros/veiculos-por-fabricante?fabricante=Toyota

Resultado esperado: 200 OK com os veículos cujo fabricante contém Toyota, incluindo o nome do fabricante.

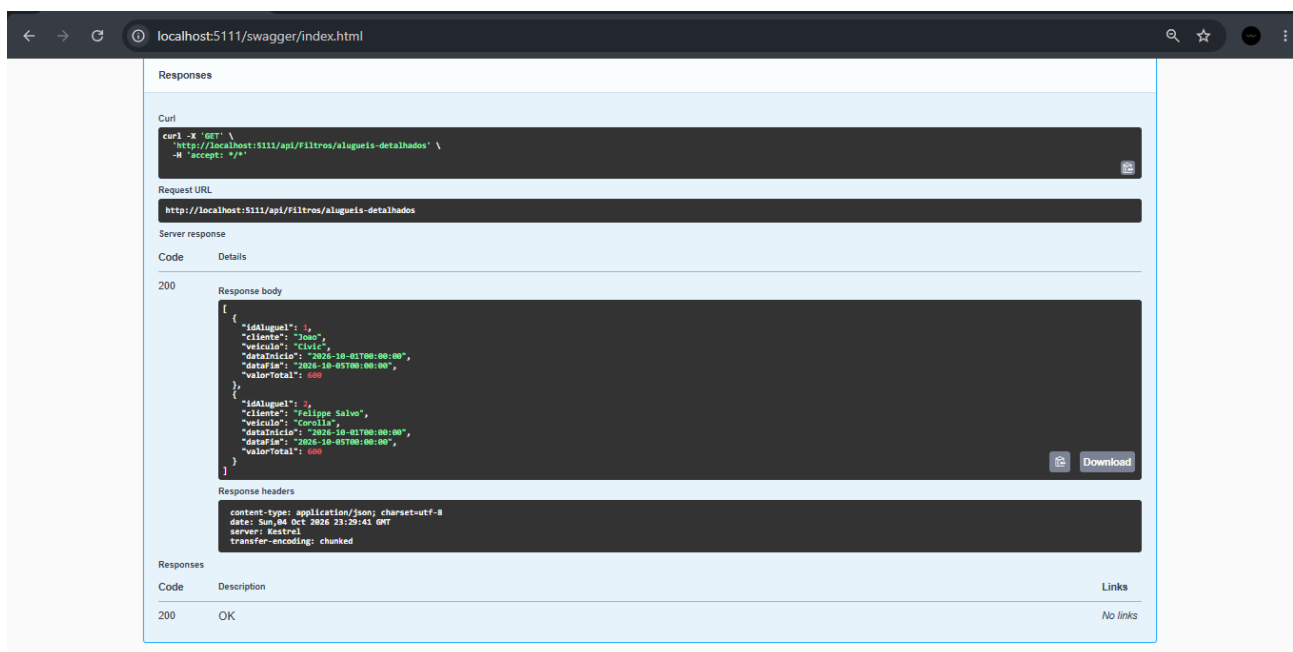


Teste 15 - Filtro: alugueis detalhados

Rota: GET /api/Filtros/alugueis-detalhados

Parâmetros/Observação: Nenhum

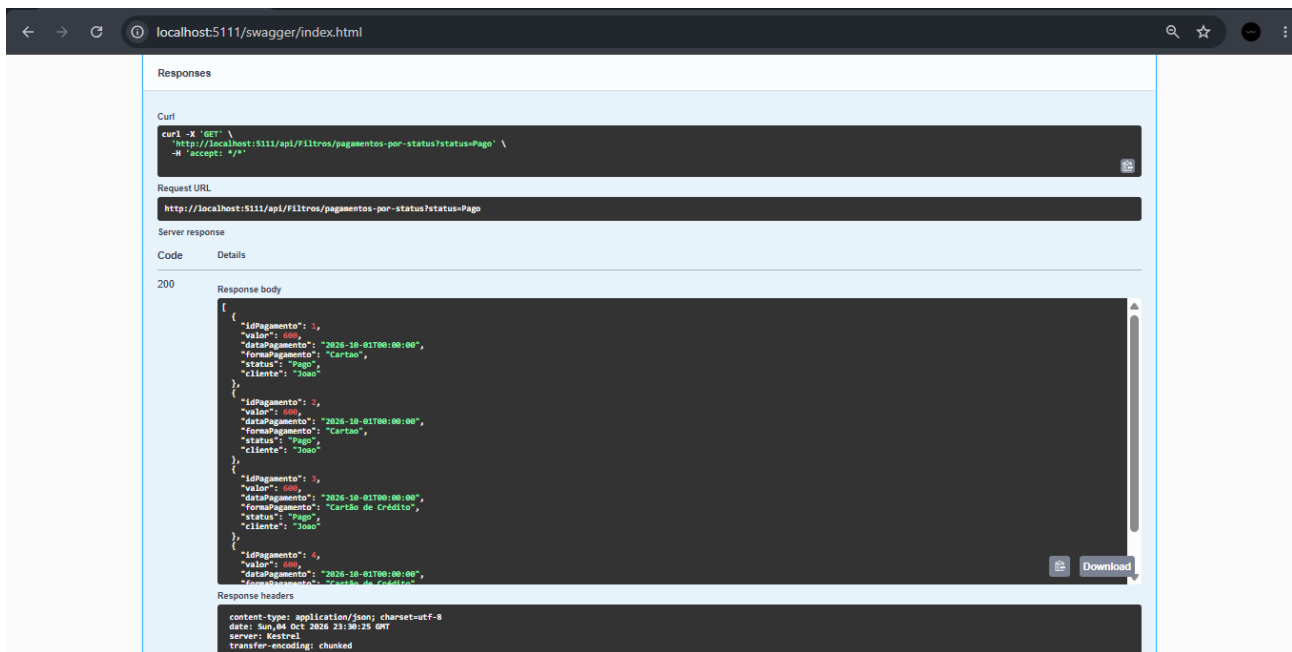
Resultado esperado: 200 OK com os alugueis incluindo nome do cliente, modelo do veículo, datas e valor total.



Teste 16 - Filtro: pagamentos por status

Rota: GET /api/Filtros/pagamentos-por-status?status=Pago

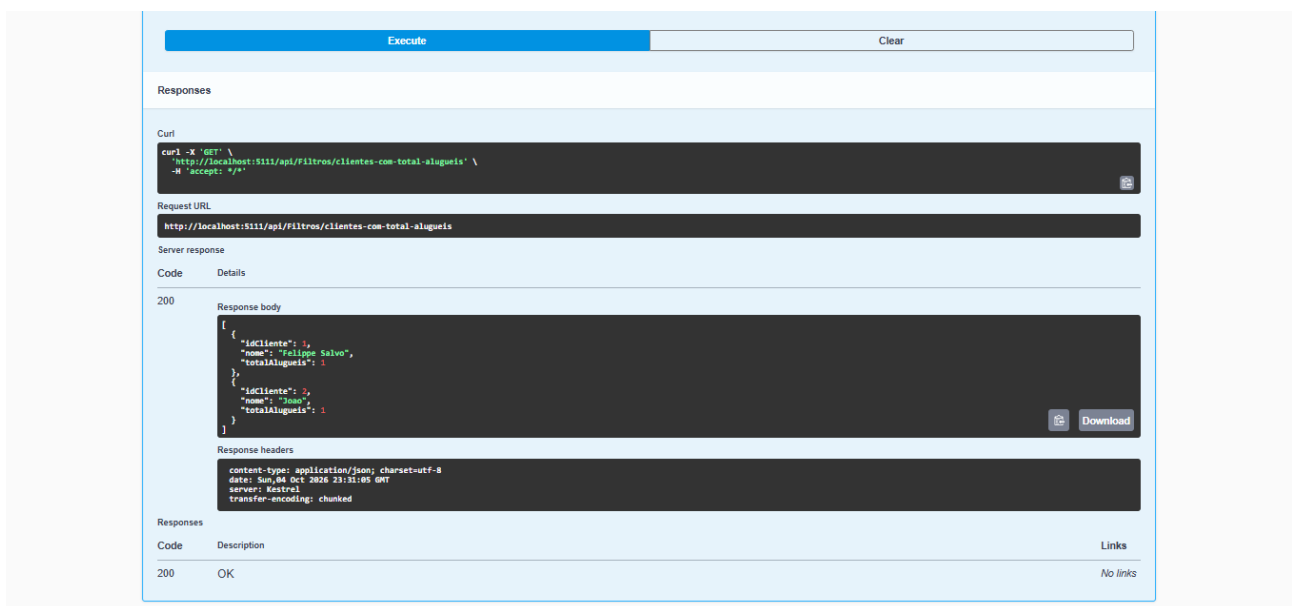
Resultado esperado: 200 OK com os pagamentos de status Pago, incluindo o nome do cliente.



Teste 17 - Filtro: clientes com total de alugueis

Rota: GET /api/Filtros/clientes-com-total-alugueis

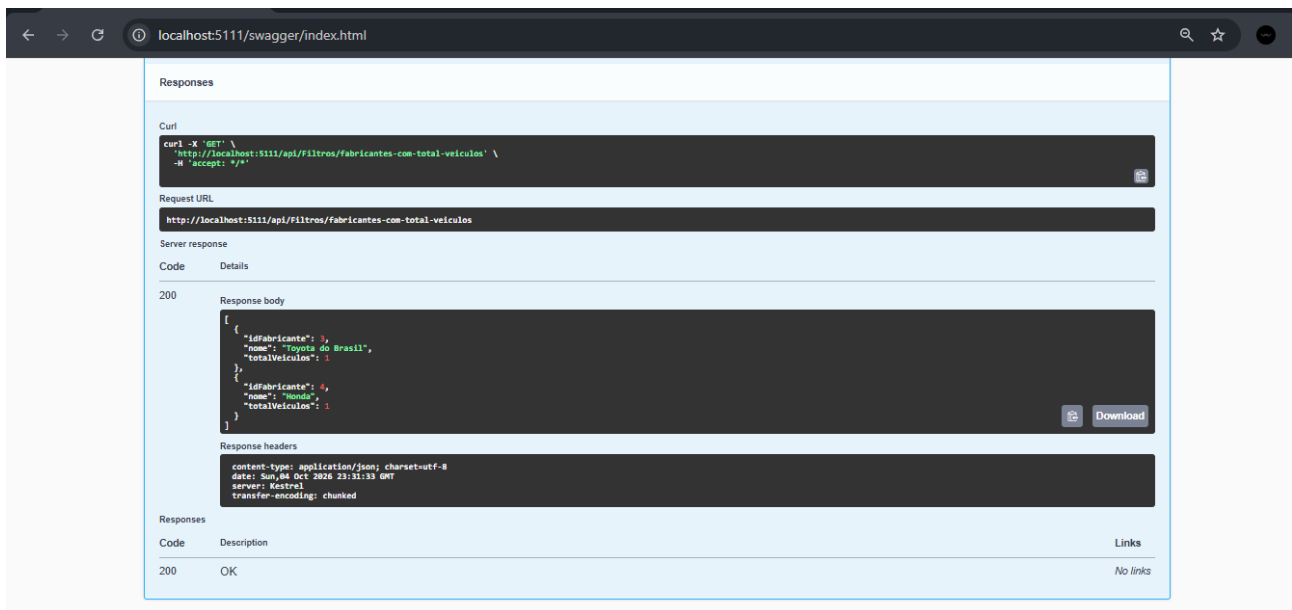
Resultado esperado: 200 OK com cada cliente e a quantidade de alugueis associados.



Teste 18 - Filtro: fabricantes com total de veículos

Rota: GET /api/Filtros/fabricantes-com-total-veiculos

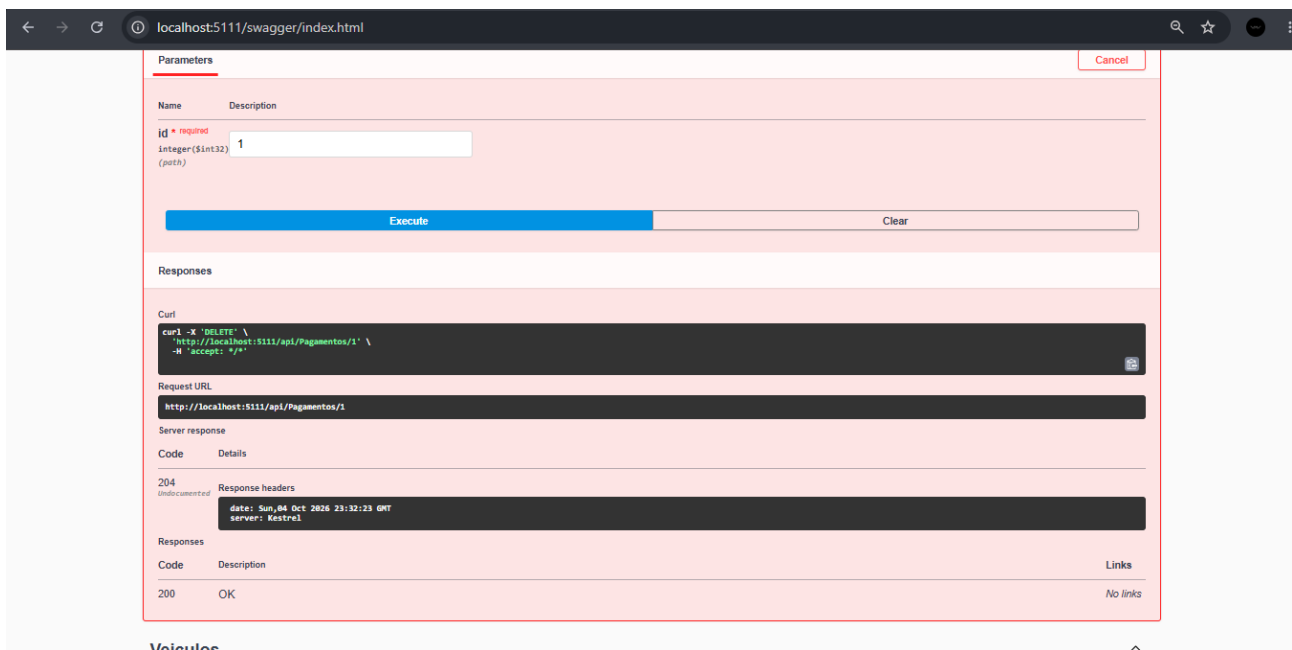
Resultado esperado: 200 OK com cada fabricante e a quantidade de veículos associados.



Teste 19 - DELETE Pagamento (remover)

Rota: DELETE /api/Pagamentos/1

Resultado esperado: 204 No Content.

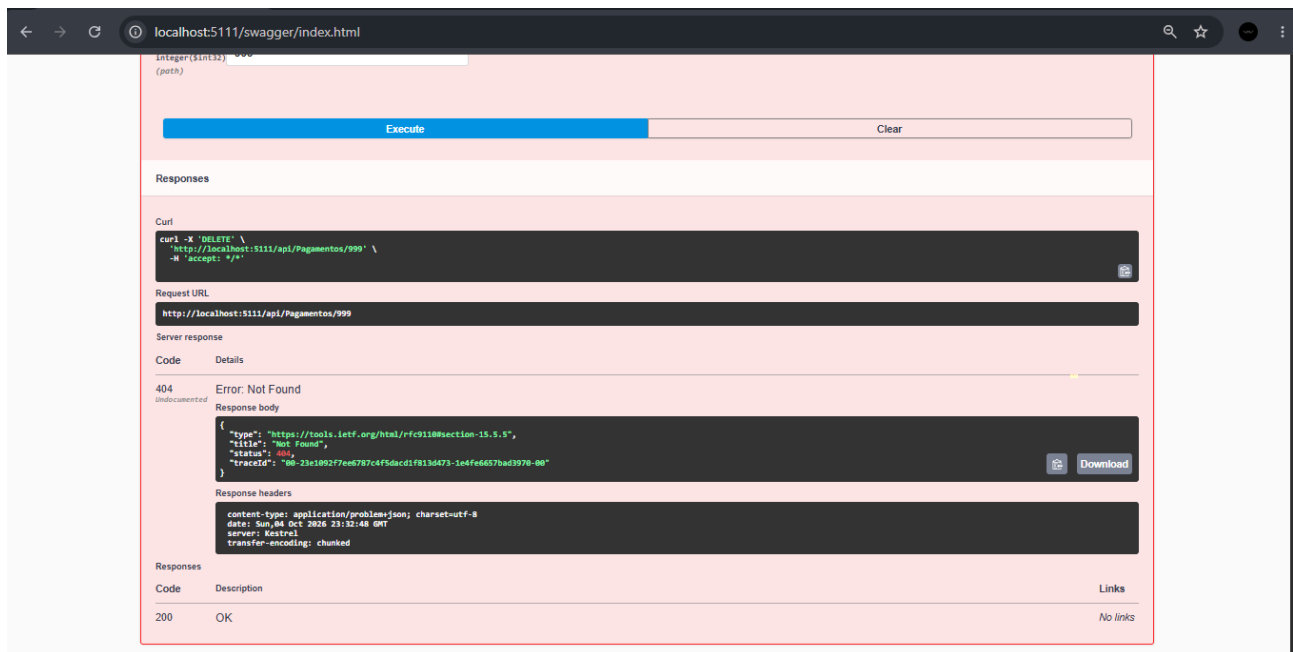


Teste 20 - DELETE registro inexistente (validação de 404)

Rota: DELETE /api/Pagamentos/999

Parâmetros/Observação: id = 999 (path)

Resultado esperado: 404 Not Found.



Conclusão

Todos os endpoints da API foram documentados e testados manualmente por meio do Swagger. As operações de CRUD (GET, POST, PUT e DELETE) das cinco entidades funcionaram conforme o esperado, retornando os códigos HTTP apropriados (200, 201, 204, 400 e 404), assim como as consultas específicas do controller de Filtros. Os testes confirmam que a API está operacional e em conformidade com os requisitos definidos.